1. ArduPilot 核心技术方案详述

ArduPilot 是目前成熟度最高、支持载具类型最丰富的方案，适合追求极致稳定性与多功能集成的工业应用。

A. 开发平台与语言 (Dev Platform & Language)

- **开发语言： C++** (核心代码) + **Lua** (脚本层)。
  - *特点：* 代码风格偏向传统嵌入式 C++，大量使用类继承。Lua 脚本提供了无需编译固件即可修改逻辑的能力。(飞控硬件选择Linux开发板可以使用python或者shell，但是在这种架构下，ArduPilot只是 Linux 系统中的一个**二进制进程**，ArduPilot 进程内部的 "Scripting" 功能依然只能用 Lua)
- **构建系统： Waf** 构建系统。
  - *特点：* 相比 Make/CMake，Waf 在处理大型依赖关系时更高效，但对新开发者有一定学习门槛。(ArduPilot 与 Waf 的深度绑定，可能**不能使用Cmake是个问题**)
- **操作系统依赖：**
  - **Target OS (飞控端)： ChibiOS** (硬实时 RTOS) 或 Linux (通过 `AP_HAL_Linux`)。
  - **Host OS (开发端)：** 原生支持 Ubuntu/WSL，Windows 下需配置 Cygwin 或 WSL 环境。

B. 核心算法与技术特征 (Key Algorithms & Features)

- **状态估计 - EKF3 (Lane Switching)：**
  - *机制：* 并行运行多组扩展卡尔曼滤波实例（Lane），监控"创新值（Innovation）"方差。
  - *作用：* **容错冗余。**当主罗盘受磁干扰或 IMU 漂移时，自动平滑切换至健康的 EKF 实例，防止"炸机"。
- **振动抑制 - 谐波陷波滤波 (Harmonic Notch Filter)：**
  - *机制：* 利用 FFT 或电调转速遥测数据，动态计算噪声频率并调整滤波器中心点。
  - *作用：* **抗噪性能。**精准滤除机架共振频率，允许 PID 参数调得更高，提升抗风性和响应速度。
- **任务逻辑 - 复杂航点与地形跟随：**
  - *机制：* 内置强大的 Mission 引擎，支持样条曲线插值导航和基于雷达的地形跟随。
  - *作用：* **工业作业。**非常适合测绘、喷洒等需要高精度轨迹控制的场景。

C. 系统特点与应用价值 (System Characteristics)

- **单体架构 (Monolithic Architecture)：**
  - 所有功能编译在一个大二进制文件中。优点是函数调用开销极小，运行效率极高；缺点是模块耦合度高，修改底层代码容易牵一发而动全身。
- **边缘计算脚本化：**
  - 支持在飞控内部运行 Lua 脚本。**作用：** 可快速实现"当电池低于 20% 且高度大于 50 米时触发特定灯光并向地面站发送警告"等定制逻辑，无需重刷固件。

D. 硬件需求 (Hardware Requirements)

- **MCU 算力：** 推荐 **STM32 H7** 系列 (480MHz+)。虽然 F4/F7 也能跑，但为了开启 EKF3 和 Lua 脚本，H7 是必须的。
- **Flash/RAM：** 极度依赖 **Flash 容量**（固件通常超过 2MB），建议 2MB Flash / 1MB RAM 起步。
- **传感器接口：** 对 SPI 总线带宽利用率极高，支持 DroneCAN (原 UAVCAN) 协议连接外设。

E. 总结

1. C++核心代码涉及大量继承等特性机制
2. Lua脚本不可代替
3. 不能使用Cmake常用编辑方案
4. 使用MCU需要H7系列，只能用HAL库，可使用Linux开发板

2.PX4 Autopilot 架构

PX4 Autopilot 是目前全球科研界与高端商业无人机领域的标杆方案，其技术基因在于高度的模块化与对现代机器人软件栈（Robotics Software Stack）的深度融合。

A. 开发平台与语言 (Dev Platform & Language)

- **开发语言： Modern C++ (C++14/17 标准)。**
  - *特点：* 代码库广泛采用面向对象设计模式与 STL 容器，相比传统嵌入式 C，其代码结构更接近现代计算机软件，利于计算机科学背景的工程师快速上手。
- **构建系统： CMake + Ninja。**
  - *特点：* 采用行业标准的 CMake 构建流，完美支持 CLion、VS Code 等现代 IDE 的代码索引、调试与重构功能，且编译速度极快。
- **操作系统依赖：**
  - **RTOS 内核：** 基于 **NuttX** 实时操作系统。
  - *特点：* NuttX 是一个完全兼容 **POSIX** 标准的嵌入式 OS。这意味着在飞控内部可以像在 Linux 中一样操作文件句柄、创建线程（Pthread）和使用 Shell 命令（NSH），提供了极佳的任务调度灵活性。

B. 核心算法与技术特征 (Key Algorithms & Features)

- **控制分配算法 (Control Allocation)：**
  - *机制：* 将"期望的力与力矩（Force & Torque）"与"电机/舵机输出（Actuator Output）"在数学上完全解耦。
  - *作用：* **构型泛化。**当开发新型 VTOL（垂直起降）或倾转旋翼机时，无需修改核心控制律，只需更改混控矩阵（Mixing Matrix）即可完成动力分配，极大地简化了复合翼机型的开发难度。
- **级联 PID 与自适应控制 (Cascaded PID & Adaptive Control)：**
  - *机制：* 采用位置环 -> 速度环 -> 姿态环 -> 角速率环的串级架构。
  - *作用：* **精准轨迹追踪。**结合自适应增益逻辑，能够有效应对飞行过程中载荷变化或电池电压下降导致的动力学模型改变，保证全包线内的飞行稳定性。
- **状态估计 - EKF2：**
  - *机制：* 使用优化的扩展卡尔曼滤波算法融合 IMU、磁力计、气压计及 GPS 数据。
  - *作用：* **鲁棒导航。**特别针对室内无 GPS 环境下的视觉定位（VIO）进行了接口优化，数据融合延迟极低。

C. 系统特点与应用价值 (System Characteristics)

- **发布-订阅中间件 (uORB Middleware)：**
  - *机制：* 系统并非单体程序，而是由数十个独立进程组成，各进程通过 **uORB** 异步消息总线通信。
  - *作用：* **故障隔离与解耦。**若"日志记录"或"遥测"模块崩溃，不会波及核心的"姿态解算"模块，极大提升了系统级安全性。同时，新增功能模块只需订阅相应话题，无需侵入原有代码。
- **ROS2 与 DDS 深度集成：**

- *机制：* 通过 **MicroXRCE-DDS** 中间件，实现飞控内部话题与 ROS2 DDS 网络的透明桥接。
- *作用：* **高带宽机载计算。**飞控可作为 ROS2 网络中的一个节点，与 Jetson Orin 等高算力计算机进行高频（>100Hz）大数据量交互。这是开发**视觉避障、SLAM（即时定位与地图构建）及多机协同编队**的首选架构。

D. 硬件需求 (Hardware Requirements)

- **MCU 算力：** 强烈推荐 **STM32 F7 / H7** 系列。
  - *原因：* NuttX 操作系统和 uORB 消息机制虽然带来了架构优势，但也引入了比 ArduPilot 更高的 CPU 开销和上下文切换损耗。
- **内存资源：** 建议 RAM ≥ 512KB，Flash ≥ 2MB。
- **接口配置：** 对以太网（Ethernet）和 USB-C 原生支持较好，适合通过高带宽总线连接机载电脑。

E. 关键参考因素 (Critical Factors)

- **商业合规性 - BSD 协议：**
  - *优势：* 采用宽松的 BSD 3-Clause 协议。允许企业修改飞控源码并集成到自己的商业产品中，**无需公开修改后的源代码。**这对于致力于保护核心算法知识产权（IP）的商业无人机公司至关重要。
- **高保真仿真 (HITL/SITL)：**
  - *优势：* 深度集成 **Gazebo** 和 **JMAVSim** 仿真器。支持完整的"软件在环"仿真，可以在虚拟环境中直接测试视觉避障算法和集群逻辑，大幅降低实机测试的炸机风险和研发成本。

F. 总结

1. 支持C++编程
2. 支持Cmake
3. 操作系统核心是NuttX，该系统支持Shell脚本(简言之类似于Ubuntu开发可使用操作文件句柄、创建线程（Pthread）和使用 Shell 命令（NSH））
4. 架构是异构计算，MCU + CPU

3.Betaflight 高性能竞速方案

Betaflight 是目前 FPV（第一人称视角）穿越机与竞速无人机领域的绝对统治者。与PX4/ArduPilot 追求"自主性"不同，Betaflight 将工程极限完全倾注于**"操控响应性"**与**"控制回路延迟极小化"**。

A. 开发平台与语言 (Dev Platform & Language)

- **开发语言： C99** (主要逻辑) + **Assembly** (底层优化)。
  - *特点：* 为了在有限的算力下压榨极致性能，核心 PID 循环和陀螺仪采样代码针对 STM32 架构进行了汇编级优化，代码风格极度精简，摒弃了复杂的类抽象。
- **运行架构： Bare Metal (裸机运行，无 RTOS)。**
  - *特点：* 不使用 ChibiOS 或 NuttX 等实时操作系统，而是采用**超级循环**（Super Loop）架构。
  - *作用：* **消除调度开销。**操作系统的时间片调度会引入微秒级的抖动（Jitter），而裸机架构能确保控制回路严格按照 8kHz（即 125μs 周期）甚至更高的频率运行，实现**确定性极高**的低延迟控制。
- **配置工具： Betaflight Configurator。**
  - *特点：* 基于 Electron 的跨平台 GUI 工具，提供极佳的用户体验，支持在线刷写固件、实时查看传感器数据和黑匣子（Blackbox）日志分析。

B. 核心算法与技术特征 (Key Algorithms & Features)

- **RPM 陷波滤波 (RPM Notch Filter)：**
  - *机制：* 通过 **双向 DSHOT** 数字协议，电调（ESC）实时回传每个电机的转速（RPM）。飞控根据实时转速计算基频及其谐波频率，动态在陀螺仪数据流上设置陷波滤波器。
  - *作用：* **手术刀式降噪。**这是一种"基于物理模型"的去噪方式，相比传统低通滤波，它几乎不引入相位延迟（Phase Latency）。这允许 PID 控制器的 D 项增益设置得极高，从而获得类似"轨道车"般的锁定感（Locked-in feel），同时避免电机过热。
- **前馈控制 2.0 (Feedforward / RC Smoothing)：**
  - *机制：* 算法不等待误差发生（P项），而是直接计算遥控杆位的**变化率（导数）**，预测飞手的意图并提前输出电机信号。
  - *作用：* **零滞后响应。**通过补偿电机的物理惯性，消除了传统 PID 控制中常见的"拖拽感"，实现了毫秒级的动态响应，使得无人机能瞬间完成翻滚等剧烈动作。
- **动态空闲 (Dynamic Idle)：**
  - *机制：* 即使在油门归零时，也强制维持电机在最低安全转速。
  - *作用：* **倒飞稳定性。**防止在剧烈机动（如自由落体、急停）中因反电动势导致电机停转（Desync），是极限花飞的保障。

C. 系统特点与应用价值 (System Characteristics)

- **极致的闭环频率：**
  - 支持 **4kHz ~ 8kHz** 的 PID 循环频率。
  - **作用：** 在时速 150km/h+ 的高速飞行中，更快的刷新率意味着系统能更早地修正气流扰动，抗风性能极强。
- **非自主导航设计：**
  - **特点：** 这是一个为了"人机合一"设计的系统，**不支持**任务航点、视觉避障或复杂的地面站控制。
  - **作用：** 所有的算力都服务于"姿态稳定"。虽然包含简单的 `GPS Rescue`（GPS 救援模式），但仅作为信号丢失后的应急返航手段，精度远低于 ArduPilot。

D. 硬件需求 (Hardware Requirements)

- **MCU 算力：** STM32 F405 / F722 / H7 系列。
  - 由于算法复杂度增加（尤其是开启 RPM 滤波和 8kHz 循环后），F3 及以下芯片已被淘汰。
- **电调协议：** 必须支持 **BLHeli_32** 或 **AM32** 固件的电调，以启用 **DSHOT600/DSHOT1200** 双向通信。
- **板载记录仪：** 极度依赖板载 Flash 芯片或 SD 卡槽，用于记录高达 4kHz 的黑匣子（Blackbox）数据，这是调参分析震动和 PID 响应的唯一依据。

E. 关键参考因素 (Critical Factors)

- **适用场景局限性：**
  - 如果您需要做**自动巡检、物流配送或二次开发（如挂载机载电脑跑 AI）**，**严禁选择** Betaflight。它没有完善的 MAVLink 支持，无法接受外部计算机的速度/位置指令。
- **开源协议： GPLv3。**
  - 代码修改需开源。但在穿越机领域，商业模式主要是卖硬件，因此软件生态非常开放活跃。
- **调参门槛：**
  - 虽然 GUI 易用，但理解滤波器设置、PID 调优以及解读黑匣子日志需要极深的专业知识，学习曲线陡峭

F. 总结

1. 仅支持C，裸机开发没有实时操作系统
2. 主控需求不高，F407就可以实现
3. 没有完善的 MAVLink 支持，无法接受外部计算机的速度/位置指令

4.iNAV 导航与长航时方案

iNAV 是 Cleanflight 代码库的一个特殊分支，其核心设计理念是在**资源受限的消费级硬件**（Resource-Constrained Hardware）上实现可靠的 GPS 自主导航功能。它牺牲了 Betaflight 极致的循环频率，换取了完整的导航栈能力。

A. 开发平台与语言 (Dev Platform & Language)

- **开发语言： C99。**
  - *特点：* 代码结构继承自 Cleanflight/Baseflight 体系，相对紧凑。近年来引入了简单的 **Logic Conditions**（逻辑编程），允许用户通过 GUI 设置简单的"If-Then"自动化逻辑（例如：高度>100m自动开启录像），无需编写代码。
- **运行架构： Bare Metal (裸机超级循环)。**
  - *特点：* 与 ArduPilot/PX4 不同，iNAV **不运行**在 RTOS（实时操作系统）上。
  - *技术权衡：* 这种设计极大地降低了 CPU 和内存开销，使其能在 STM32 F405 等低端芯片上流畅运行完整的导航算法。但缺点是任务调度的确定性不如 RTOS，处理复杂并发任务（如同时进行高频数传和各种外设通信）时可能会出现资源竞争。
- **配置工具： iNAV Configurator。**
  - *特点：* 图形化界面直观，特别是其 **Mission Planner** 标签页，虽然不如 ArduPilot 强大，但足以完成基本的航点规划和电子围栏设置。

B. 核心算法与技术特征 (Key Algorithms & Features)

- **轻量化位置估算器 (Lightweight Position Estimator)：**
  - *机制：* 采用优化的互补滤波（Complementary Filter）与简化版卡尔曼滤波算法。
  - *作用：* **算力适配。**能够在不依赖双精度浮点运算单元（FPU）的情况下，稳定融合 GPS（位置/速度）、气压计（高度）、磁力计（航向）和光流数据。虽然绝对精度略逊于 ArduPilot 的 EKF3，但在 F4/F7 芯片上实现了极佳的效能功耗比。
- **固定翼自动起飞 (Fixed-Wing Auto Launch)：**
  - *机制：* 结合加速度计阈值检测与姿态控制律。
  - *作用：* **单兵作业辅助。**飞控检测到"投掷"动作（高 G 值加速度）后，延迟数毫秒自动启动电机并锁定爬升姿态（Pitch Angle）。这解决了单人手抛大型固定翼飞机时"一手抛机、一手控杆"的操作难题，极大降低了坠机风险。
- **混合 PID 控制器 (Navigation-Augmented PID)：**
  - *机制：* 重写了 PID 核心，引入了 **Heading Hold（航向锁定）** 和 **Turn Assistant（转弯辅助）**。
  - *作用：* 针对长航时飞行优化。在 GPS 模式下，PID 控制器主要服务于位置保持；在手动模式下，它又能提供接近穿越机的平滑手感。

C. 系统特点与应用价值 (System Characteristics)

- **MSP 协议与 CMS 交互系统：**

    - *机制：* 深度利用 **MSP (MultiWii Serial Protocol)** 协议。
    - *作用：* **沉浸式人机交互。**iNAV 允许飞手直接通过 FPV 视频眼镜的 **OSD（屏幕显示）菜单** 修改 PID 参数、VTX 功率甚至重置航点任务，无需连接电脑。这种特性非常契合野外单人操作 FPV 远航的场景。
- **多构型支持：**
    - 除了标准的多旋翼和固定翼，iNAV 对 **Flying Wing（飞翼）** 和 **V-Tail（V尾）** 构型的混控支持非常完善，且设置比 ArduPilot 简单直观。

D. 硬件需求 (Hardware Requirements)

- **MCU 算力：** 最低 STM32 F405，推荐 **STM32 F722 / H7。**
    - 虽然早期的 F3 芯片曾被支持，但随着导航算法复杂化，目前已不再推荐。
- **传感器配置： 必须**配备带有磁力计（罗盘）的 GPS 模块和气压计。
    - 与 Betaflight 不同，iNAV 若缺失罗盘，无法进入任何 GPS 辅助模式。
- **成本优势：** iNAV 通常运行在廉价的穿越机飞控（AIO板卡）上，整套飞控成本通常仅为 Pixhawk 系列的 1/3 到 1/4。

E. 关键参考因素 (Critical Factors)

- **安全性限制：**
    - *局限：* 缺乏 ArduPilot 那样完善的**冗余机制**（如双 GPS 切换、多 EKF 核心）。
    - *建议：* **不建议**用于城市上空人群密集区飞行或负载昂贵测绘设备的行业应用。它更适合 FPV 爱好者的长距离探险（Long Range FPV）或低成本农业侦察。
- **调参体验：**
    - 相比 ArduPilot 的"参数海洋"，iNAV 的参数逻辑更简化，但在固定翼调参上，需要飞手具备一定的 PID 理论基础，自动调参（Auto-Tune）功能的稳定性略逊于 ArduPilot。

F. 总结

1. 仅支持C，裸机开发没有实时操作系统
2. 主控需求不高，F407就可以实现

5.Paparazzi UAV 模块化方案

在技术选型报告中，Paparazzi UAV 应被定位为"**学术科研、非标气动布局验证及多机协同**"的专用方案。它与 ArduPilot/PX4 的最大区别在于其"代码生成"的设计哲学。 Paparazzi UAV 是由法国国家民航大学（ENAC）主导维护的开源飞控项目，其技术路线与主流商业飞控截然不同。它不追求"开箱即用"的通用性，而是专注于**极高的可配置性、形式化验证（Formal Verification）以及分布式多机协同**，是学术界验证新型控制理论和非传统气动布局的首选平台。

A. 开发平台与语言 (Dev Platform & Language)

- **开发语言： C (核心)** + **OCaml** (生成器) + **XML** (配置)。
    - *特点：* 这是一个基于 **"模型驱动开发（Model-Based Design）"** 理念的系统。开发者并不直接修改 C 代码中的参数，而是编写 XML 配置文件来描述机身几何结构、传感器类型和控制逻辑。编译系统会解析这些 XML，自动生成针对该特定无人机的静态 C 代码。
- **构建系统： Custom Generators。**

- *特点：* 这种"静态编译"机制意味着固件中只包含该次任务所需的代码，没有多余的"通用逻辑"或动态判断。
- *作用：* **极致的资源效率。**生成的二进制文件极小且运行效率极高，能够在计算资源极度受限的微型 MCU 上运行复杂的控制律。

- **通信架构： Ivy Software Bus。**
  - *机制：* 一种基于文本的轻量级软件总线中间件。
  - *特点：* 不同于 MAVLink 的点对点模式，Ivy 允许地面站、模拟器、以及飞行进程以"广播"形式交换数据。这使得多架无人机之间的通信（Swarming）如同局域网聊天一样简单，无需复杂的路由配置。

B. 核心算法与技术特征 (Key Algorithms & Features)

- **飞行计划状态机 (XML Flight Plans)：**
  - *机制：* 飞行任务不仅仅是经纬度航点的列表，而是一个完整的**状态机（State Machine）**。
  - *作用：* **逻辑编程。**用户可以在 XML 中编写复杂的条件逻辑（例如："如果电池 < 30% 且距离 > 1km，则跳转到 Block A；否则执行 Block B"）。这种在 XML 层面定义的任务逻辑经过形式化验证，安全性远高于脚本语言。
- **非线性控制律 (Advanced Control Loops)：**
  - *机制：* 除了标准的 PID，Paparazzi 原生支持 **INDI (Incremental Nonlinear Dynamic Inversion)** 等先进非线性控制算法。
  - *作用：* **学术研究。**非常适合解决扑翼机、尾座式垂直起降（Tailsitter）等具有强非线性动力学特征的控制难题。
- **原生多机协同 (Native Swarming)：**
  - *机制：* 利用 Ivy 总线，多架飞机可以共享彼此的位置和状态。
  - *作用：* **编队飞行。**无需额外的机载电脑或 ROS 节点，仅凭 MCU 即可实现简单的编队跟随和避撞逻辑。

C. 系统特点与应用价值 (System Characteristics)

- **硬件无关性 (Hardware Agnostic)：**
  - 由于驱动代码也是通过 XML 定义生成的，Paparazzi 移植新硬件非常容易。它既可以运行在 STM32 裸机上，也可以运行在 Linux (Raspberry Pi/Gumstix) 甚至 x86 模拟器上。
- **混合系统支持：**
  - 支持异构系统协作。例如，地面站可以同时监控一架固定翼、两架多旋翼和一个地面车辆，并通过 Ivy 总线协调它们的行动。

D. 硬件需求 (Hardware Requirements)

- **MCU 算力：** 极低。STM32 F4 甚至更老的芯片即可流畅运行。
- **外设支持：** 官方硬件（如 Krooz, Chimera）集成了极其丰富的外设接口，但对通用飞控板（如 Pixhawk）的适配需要一定的底层驱动调试能力。

E. 关键参考因素 (Critical Factors)

- **陡峭的学习曲线 (Steep Learning Curve)：**
  - *风险：* 它的配置方式类似 Linux 内核编译，需要开发者理解 XML 语法、编译原理和控制理论。缺乏像 Mission Planner 那样傻瓜式的"点击-设置"界面。
  - *结论：* **不适合**普通商业应用开发或初级用户。

- **社区生态：**
  - 社区规模较小，主要集中在欧洲的高校和研究所（如 TU Delft, ENAC）。遇到问题时，往往需要直接阅读源码或在邮件列表中求助，响应速度不如 ArduPilot/PX4 社区。

F. 总结

- **C (核心)** + **OCaml** (生成器) + **XML** (配置)，类似于打驱动
- 硬件平台不受限，可使用MCU也可以使用CPU
- 配置方式类似 Linux 内核编译，需要开发者理解 XML 语法、编译原理和控制理论

6. 总结

维度	ArduPilot	PX4 Autopilot	Betaflight	iNAV	Paparazzi UAV
定位	工业应用、极致稳定、多功能集成	科研界标杆、高端商业、机器人融合	FPV 竞速、极致响应、低延迟控制	长航时 FPV、轻量级导航、高性价比	学术验证、异构集群、非标气动布局
开发语言	**C++** (核心) + **Lua** (脚本) *大量使用类继承*	**Modern C++ (14/17)** *面向对象，STL容器*	**C99** + **汇编** *极致精简*	**C99** *引入逻辑条件编程*	**C** (核心) + **OCaml** + **XML** *模型驱动开发*
构建系统	**Waf** *难以替换为 CMake，IDE 门槛较高*	**CMake + Ninja** *原生支持现代 IDE，构建极快*	**Makefile** 等 *裸机编译*	**Makefile** 等 *裸机编译*	**Custom Generators** *静态代码生成*
OS 架构	**ChibiOS** (RTOS) 或 Linux 进程 *单体架构 (Monolithic)*	**NuttX** (POSIX RTOS) *模块化架构 (uORB 中间件)*	**Bare Metal** (裸机) *超级循环，无调度开销*	**Bare Metal** (裸机) *超级循环*	**硬件无关** *可跑在 MCU 裸机或 Linux 上*
硬件需求	**STM32 H7** (推荐) *需 2MB+ Flash 跑 Lua/EKF3*	**STM32 F7/H7** *上下文切换开销较大*	**STM32 F405/F7/H7** *高性能算力用于滤波*	**STM32 F405** (最低) *资源受限硬件上运行*	**极低** (F4 即可) *生成的二进制极小*
核心算法	**EKF3** (多核冗余切换)、谐波陷波滤波、复杂任务引擎	**EKF2** (视觉融合强)、控制分配 (Control Allocation)、自适应 PID	**RPM 陷波滤波**、前馈控制 2.0、动态空闲	**轻量级位置估算**、固定翼自动起飞、混合 PID	**状态机逻辑 (XML)**、非线性控制律 (INDI)、原生多机协同

维度	ArduPilot	PX4 Autopilot	Betaflight	iNAV	Paparazzi UAV
扩展/生态	**Lua 脚本** (边缘计算)、Linux 下作为进程运行	**ROS2 / DDS** 深度集成、异构计算 (MCU+CPU) 首选	**DSHOT 双向通信**、黑匣子日志分析	**MSP 协议** (眼镜端调参)、无需代码的逻辑编程	**Ivy 总线** (分布式通信)、形式化验证
协议/合规	**GPLv3** *修改必须开源，商业受限*	**BSD 3-Clause** *可闭源商用，保护 IP*	**GPLv3**	**GPLv3**	**GPLv2** (主要)

- **选 ArduPilot：** 如果需要一个"装上就能干活"的系统，用于测绘、农业喷洒或复杂的工业巡检，且不介意 GPL 协议的开源限制。其 EKF3 的冗余机制是目前最稳健的防炸机方案。
- **选 PX4：** 如果在做机器人研究（SLAM、避障），或者开发需要保护知识产权的商业产品。PX4 的模块化设计和 ROS2 支持使其成为连接 AI 计算机（如 Jetson）的最佳管道。
- **选 Betaflight：** 仅限于竞速或花飞。如果需要绝对的锁定感和毫秒级响应，不要考虑带有导航栈的系统。
- **选 iNAV：** 如果想组装一台低成本的长航时固定翼飞机去拍风景，硬件只需 F4 芯片，成本极低且具备返航功能。
- **选 Paparazzi：** 除非需要研究扑翼机控制理论或大规模集群算法，否则不建议触碰，其 XML 编译的学习曲线极陡峭。